SPARXSTAR

Sirus

Context Engine Specification

Version 3.0 · Starisian Technologies

`Namespace: Starisian\Sparxstar\Sirus · Repository: Starisian-Technologies/sparxstar-sirus`

PHP 8.2+ · WordPress 6.8+ · PSR-4 · PSR-12 · PHPStan Level 5 · strict typing

v3.0 absorbs all open items from v2.0 that are now closed per the Platform Integrity Map v1.0.

CONFIDENTIAL · PATENT PENDING

1. What Sirus Is

Sirus is the context engine of the Sparxstar platform. Before identity is established, before authentication runs, before any application logic executes — Sirus establishes the environment. It determines what is happening right now: who is present, on what device, in what environment, under what authority.

Sirus produces context. It does not make authorization decisions. That is Helios. Sirus does not enforce governance. That is Méh₁n̥s.

System	Answers
Sirus	What is the situation? Who is present, on what device, in what environment?
Helios	Given the situation, does this specific request proceed?
Méh₁n̥s	Given the context and proof, does this action comply with governance?

Helios proves the right to proceed. Sirus carries the context that makes that proof meaningful.

1.1 Deployment — Sirus Is the Platform Kernel

Sirus must be deployed as a WordPress Must-Use (mu-plugin). It loads before any standard plugin, theme, or authentication hook can execute. It cannot be deactivated by any site administrator.

The boot order is enforced by mu-plugins/00-sparxstar-loader.php, which requires Sirus first. See the Platform Integrity Map v1.0, Rule 2.1 and Rule 2.4 for the full boot sequence.

```
// mu-plugins/00-sparxstar-loader.php
require_once __DIR__ . '/sparxstar-sirus/sparxstar-sirus.php';
require_once __DIR__ . '/sparxstar-helios/sparxstar-helios.php';
// Sky, Méh₁n̥s, Dheghom follow in order
```

2. What Sirus Is Not

- Not the authentication system — that is Helios
- Not the governance engine — that is Méh₁n̥s
- Not the storage layer — that is Dheghom
- Not the source of policy decisions — Sirus describes; Helios and Méh₁n̥s decide
- Not a WordPress session manager for frontend users — WordPress sessions belong to wp-admin only
- Not the source of identity claims in the ContextPulse — the Pulse carries device state, trust score, behavior flags, geo zone, and session duration only; identity resolution requires the full SirusContext or a Helios-verified request context

3. Standalone Operational Rule

Sirus must operate correctly as part of the full multi-repo system and must also be capable of operating on its own. When operating with the other repos present, it behaves as designed within the full architecture. When operating on its own, it remains functional to the highest capability possible without providing the full security, governance, or sovereignty guarantees available in the complete system.

Standalone Sirus can resolve environment and device context. Without Helios, it cannot enforce agreement. Without Mḗhiṇs, it cannot enforce governance.

4. The Execution Model

Sirus inverts the standard WordPress model. The order of operations:

```
Environment  → what is happening physically and technically
  ↓
Device       → which device, is it known, is it drifting
  ↓
Identity     → who is this, at what tier of recognition
  ↓
Authority    → under whose jurisdiction are they acting
  ↓
Capability   → what are they permitted to do
  ↓
Application  → feature code runs here, and only here
```

Device continuity precedes identity. A contributor is recognized by their device before they are ever known by name.

5. CLI and Non-HTTP Context

WP-CLI and background cron jobs run outside the HTTP request lifecycle. There is no device fingerprint, no ContextPulse, and no browser context. ContextEngine::current() must not throw in these environments.

When PHP_SAPI === "cli", Sirus resolves a System-Level Context with fixed values:

```
identity_id  = "SYSTEM"
trust_score  = 1.0
trust_level  = "NORMAL"
authority_id = "GLOBAL"
device_id    = "CLI"
```

Helios grants ALLOW_ORIGIN to the SYSTEM identity for all maintenance tasks, bypassing proof and Pulse verification. This bypass is available exclusively when identity_id === "SYSTEM" and PHP_SAPI === "cli". The CryptographicAuditLedger still records all writes under the SYSTEM identity.

6. The Core Context DTO

The SirusContext is the primary output of the context engine. Everything downstream consumes it.

```
final class SirusContext
{
    public const CONTEXT_VERSION = 3;

    public function __construct(
        public readonly string  $context_id,
        public readonly string  $environment_id,
        public readonly string  $network_id,
        public readonly string  $site_id,
        public readonly string  $device_id,
        public readonly string  $session_id,
        public readonly ?string $identity_id,
        public readonly ?string $authority_id,
        public readonly array   $aggregated_authorities,
        public readonly array   $role_set,
        public readonly array   $capabilities,
        public readonly string  $trust_level,
        public readonly float   $trust_score,
        public readonly array   $cryptographic_signatures,
        public readonly int     $issued_at,
        public readonly int     $expires,
    ) {}
}
```

aggregated_authorities: a user may belong to multiple sovereign authorities simultaneously. All applicable authorities are resolved and held. The Sieve evaluates against all of them.

trust_score: derived at resolution time, never stored. Recomputed on each context build. See Section 11 for the computation algorithm.

identity_id: nullable. Anonymous and device-tier contexts carry no identity_id. Callers must use the null-safe operator (?->) when consuming SirusContext.

7. The Environment Record DTO

```
final class EnvironmentRecord
{
    public function __construct(
        public readonly string $environment_id,
        public readonly string $browser_name,
        public readonly string $browser_version,
        public readonly string $os,
        public readonly string $os_version,
        public readonly string $device_type,
        public readonly string $device_brand,
        public readonly string $device_model,
        public readonly string $network_effective_type,
        public readonly string $ip_address,
        public readonly array  $location,
        public readonly string $time_zone,
        public readonly bool   $is_bot,
        public readonly int    $captured_at,
```

```
    ) {}
}
```

Privacy rules enforced at this DTO: IP addresses stored with last octet zeroed (192.168.1.0). Location is region-level only: country, region, approx_lat, approx_lng. Exact coordinates are never stored without explicit per-session grant. At anonymous and device identity tiers, no PII is captured or inferred.

8. The Device Record DTO

```
final class DeviceRecord
{
    public function __construct(
        public readonly string $device_id,
        public readonly string $fingerprint_hash,
        public readonly string $environment_json,
        public readonly int    $first_seen,
        public readonly int    $last_seen,
        public readonly string $trust_level,
        public readonly bool   $is_active,
    ) {}

    public function isActive(): bool
    {
        $ttl = (int) apply_filters('sparxstar_sirus_device_ttl_days', 90);
        return (time() - $this->last_seen) < ($ttl * DAY_IN_SECONDS);
    }
}
```

The device_id is always server-issued. The JS fingerprint is probabilistic metadata for drift detection only. Fingerprints change; server-issued IDs persist.

fingerprint_hash formula: sha256(visitorId + userAgent + ipSubnet). visitorId is a client-side persistent identifier derived from JS signal capture (canvas hash, screen resolution, timezone, user agent). It is used exclusively for drift detection and is not a security primitive.

9. Identity Resolution

Sirus recognizes five identity tiers resolved in ascending order of trust:

```
anonymous_identity   → no device recognition, no session
device_identity      → known device, no login
contributor_identity → device-known, contributor record attached
user_identity        → authenticated via Helios agreement
authority_identity   → governance-level actor
```

When resolution beyond device identity is required, Sirus checks its own trust cache first. Cache key: helios_trust:{device_id}:{session_id}, TTL 120 seconds. This eliminates the majority of Helios calls while keeping trust resolution fresh.

The trust cache requires a persistent object cache backend (Redis or Memcached). In environments without a persistent object cache, the trust cache layer is disabled and every request falls back to per-request pulse verification. Transients must not be used for trust state.

Sirus must never call wp_set_auth_cookie(), issue JWTs, or replicate edge security logic.

10. The Context Pulse

The Context Pulse is the most important output Sirus produces. It is the signed, short-lived, request-scoped statement that Helios consumes at the edge to make agreement decisions without booting the full stack.

```
final class ContextPulse
{
    public function __construct(
        public readonly string $pulse_id,
        public readonly string $device_id,
        public readonly float  $trust_score,
        public readonly string $trust_level,
        public readonly array  $behavior_flags,
        public readonly string $geo_zone,
        public readonly int    $session_duration,
        public readonly int    $issued_at,
        public readonly int    $expires_at,
        public readonly string $signature,
    ) {}
}
```

What the pulse contains: enough for Helios to evaluate agreement without a database lookup — device state, trust score, trust level, behavior flags, geo zone, session duration.

What the pulse does not contain: identity claims, authority decisions, user data. The pulse describes the device and context state only. Any implementation that reads identity_id from the Pulse is incorrect.

Travels as: HttpOnly, SameSite=Strict cookie. JavaScript cannot access it. XSS cannot steal it. Signed with HMAC-SHA256 using the Sirus signing key. Helios verifies this signature before trusting any pulse content.

11. Pulse TTL Strategy

The pulse TTL varies based on context quality and resource sensitivity requirements. The pulse is a lease, not a password. TTL values must be configurable via environment variable or WordPress filter — they must not be hardcoded.

Context	Working Default TTL
High-connectivity, stable	60–120 seconds
Low-connectivity (2G, rural) — Level 1 only	Up to 10 minutes
Medium sensitivity (Level 2)	30–60 seconds

High sensitivity / sovereign (Level 3)	15–30 seconds

[OPEN] Exact TTL values per context tier and connectivity profile require real-world testing in African low-connectivity environments. The values above are working defaults, not final values.

12. Trust Engine

The TrustEngine computes the trust score from available signals. The score is derived at resolution time and never stored. The following algorithm is the Standard Default for v1.0 per the Platform Integrity Map Decision 5.1.

12.1 Trust Score Computation

```
final class TrustEngine
{
    public function compute(
        DeviceRecord     $device,
        EnvironmentRecord $env,
        SessionState     $session
    ): float {
        $score = 1.0;

        if ($device->isDrifting())        { $score -= 0.3; } // device continuity
        if ($env->isGeoMismatch())         { $score -= 0.2; } // geo consistency
        if ($session->isNew())             { $score -= 0.1; } // session stability
        if ($session->hasRecentFailures()) { $score -= 0.3; } // anomaly flags

        return max(0.0, min(1.0, $score));
    }
}
```

Signal weights: device continuity 30% of deduction (drift detected mid-session, some drift expected on mobile networks), geo consistency 20%, session stability 10%, anomaly/recent failures 30%. Total maximum deduction: 0.9. Minimum possible score: 0.1 (all signals failing simultaneously).

12.2 Trust State

```
NORMAL           → context stable, no anomalies
STEP_UP_REQUIRED → anomaly detected, additional proof needed
LOCKED           → serious anomaly or repeated violations, context suspended
```

Signal	Result
Device fingerprint mismatch mid-session	STEP_UP_REQUIRED
ASN / country jump within short window	STEP_UP_REQUIRED
Multiple failed authentication bursts	STEP_UP_REQUIRED
Repeated anomalies	LOCKED
Session hijack pattern detected	LOCKED
Rapid device shifting	STEP_UP_REQUIRED
Geographic impossibility	LOCKED

State transitions: NORMAL → STEP_UP_REQUIRED → LOCKED. LOCKED → NORMAL requires verified recovery only. No silent downgrade recovery. LOCKED requires explicit, verified action to clear.

13. PulseVerifier — Canonical Verification Contract

The PulseVerifier class defines the canonical six-check verification sequence. This contract is shared between Sirus (which generates pulses) and Helios (which verifies them). Both edge and origin implementations must produce identical results for identical inputs.

```
final class PulseVerifier
{
    public function verify(ContextPulse $pulse): VerificationResult
    {
        // 1. Signature validation (HMAC-SHA256)
        $expected = hash_hmac(
            'sha256',
            $pulse->canonicalString(),
            $this->sirusSigningKey
        );
        if (!hash_equals($expected, $pulse->signature)) {
            return VerificationResult::TAMPERED;
        }

        // 2. Expiration check
        if (time() > $pulse->expires_at) {
            return VerificationResult::EXPIRED;
        }

        // 3. Issued-at sanity (no future skew beyond tolerance)
        if ($pulse->issued_at > time() + 30) {
            return VerificationResult::MALFORMED;
        }

        // 4. device_id presence and format validation
        if (empty($pulse->device_id) || !preg_match('/^[0-9a-f-]{36}$/i',
$pulse->device_id)) {
            return VerificationResult::MALFORMED;
        }

        // 5. trust_level allowed enum
        if (!in_array($pulse->trust_level, ['NORMAL', 'STEP_UP_REQUIRED',
'LOCKED'], true)) {
            return VerificationResult::MALFORMED;
        }

        // 6. trust_score bounds (0.0 - 1.0)
        if ($pulse->trust_score < 0.0 || $pulse->trust_score > 1.0) {
            return VerificationResult::MALFORMED;
        }

        return VerificationResult::VALID;
    }
}
```

VerificationResult enum values: VALID, EXPIRED, TAMPERED, MALFORMED. Any result other than VALID must cause Helios to deny the request immediately.

14. Device Continuity

14.1 First Visit

```
JS client sends fingerprint signals to /sparxstar/v1/device
  ↓
Server parses UA via Matomo DeviceDetector (PHP)
fingerprint_hash = sha256(visitorId + userAgent + ipSubnet)
  ↓
Server creates device_id = uuid_v4(), stores DeviceRecord
  ↓
Server returns device_id to browser
Browser stores: localStorage.setItem('sirus_device_id', device_id)
```

14.2 Session Recovery (localStorage Cleared)

```
JS client sends fingerprint metadata without device_id
  ↓
Server attempts fingerprint match against known records
  via DeviceMatcher (see Section 14.3)
If STRONG_MATCH    → restore device record, issue new device_id
If WEAK_MATCH      → restore with STEP_UP_REQUIRED
If NO_MATCH        → treat as new device, require step-up
```

14.3 DeviceMatcher — Explicit Scoring Thresholds

Fingerprint recovery requires explicit scoring. The following thresholds are the normative standard per Platform Integrity Map Decision 5.4.

```
final class DeviceMatcher
{
    public function match(array $candidateSignals, DeviceRecord $record):
MatchResult
    {
        $score = 0.0;

        if ($candidateSignals['user_agent'] === $record->userAgent)    { $score +=
0.4; }
        if ($candidateSignals['ip_subnet']  === $record->ipSubnet)     { $score +=
0.3; }
        if ($candidateSignals['device_type']=== $record->deviceType)   { $score +=
0.2; }
        if ($candidateSignals['timezone']   === $record->timezone)     { $score +=
0.1; }

        if ($score >= 0.8) { return MatchResult::STRONG_MATCH; }
        if ($score >= 0.6) { return MatchResult::WEAK_MATCH_REQUIRE_STEP_UP; }
        return MatchResult::NO_MATCH;
    }
}
```

STRONG_MATCH (≥0.8): device record restored normally. WEAK_MATCH (≥0.6): device restored with STEP_UP_REQUIRED trust state. NO_MATCH (<0.6): treated as new device, full step-up required.

14.4 Cross-Domain Continuity (Multisite / Subdomains)

localStorage is origin-scoped. On multisite installations with subdomains, each subdomain has its own localStorage. The Network Context Broker handles cross-origin device continuity.

When a user navigates from one origin to another, the source origin generates a short-lived signed context token (30-second TTL) passed as a URL parameter. The destination origin validates the token, restores context, writes the device_id to its own localStorage, and removes the token from the URL via history.replaceState(). The device is re-established on the new origin using the signed token as proof of continuity.

14.5 Device Shifting Behaviour

Scenario	Sirus Evaluation	Helios Action
Known device	Valid signed fingerprint, matches history	Agreement proceeds normally
New device (first time)	No existing fingerprint for this identity	Step-up required, limited access until verified
Rapid device shifting	Multiple new fingerprints in short duration	Anomaly flagged, Level 2+ denied pending review
Stolen device (reported)	Admin kicks device via admin UI	KV revocation written, edge denies globally

15. Step-Up Policy

The StepUpPolicy class determines when additional proof is required. This logic must be evaluated before returning a STEP_UP AgreementResult. It is owned by Helios but defined here because it depends on trust state values computed by Sirus.

```
final class StepUpPolicy
{
    public function requiresStepUp(
        ContextPulse      $pulse,
        ResourceSensitivity $level
    ): bool {
        if ($pulse->trust_level === 'STEP_UP_REQUIRED') {
            return true;
        }
        if ($level === ResourceSensitivity::LEVEL_2 && $pulse->trust_score < 0.7) {
            return true;
        }
        if ($level === ResourceSensitivity::LEVEL_3) {
            return true; // Level 3 always requires step-up — every request, every
time
        }
        return false;
    }
}
```

16. Consent Lifecycle

Sirus manages consent at three levels. Consent state is part of SirusContext and governs what actions are permitted regardless of identity tier.

- Technical consent (before login): cookies, fingerprint, session storage. No session may be created before technical consent is recorded.
- Purpose consent (per-action): recording audio, sharing data, publishing, exporting. Each purpose requires explicit confirmation, recorded per purpose per session.
- Consent is device-level, session-level, user-level (when identity established), and purpose-level. Consent history is immutable. Revocation is respected immediately. Integrated with WordPress privacy export/erase.

17. Authority Resolution

Authority is a sovereign governance domain — not a WordPress role. Sirus resolves which authority or authorities apply before any feature code runs.

Authority types: starisian, aiwa, sparxstar_network, tribal_authority, partner_institution.

Multi-authority membership: a user may belong to multiple authorities simultaneously. All applicable authorities are resolved and held in aggregated_authorities. The Sieve evaluates rules from all applicable jurisdictions and applies the most restrictive outcome where conflicts exist. Authority can change mid-session cleanly via context token update, enabling elder attestation, youth-level gating, and jurisdiction-aware behaviour without rebuilding the application stack.

18. The Context Pulse REST Endpoint

A dedicated endpoint allows Helios and other authorized consumers to request a fresh pulse. The endpoint is not used by the browser JS client to establish initial context — initial context is established by the mu-plugin at boot before WordPress processes any request. The endpoint is specifically for Helios origin-side calls and server-to-server re-verification.

```
POST /sparxstar/v1/pulse
Request:  { device_id, resource_sensitivity, request_id }
Response: signed ContextPulse as HttpOnly cookie
          + pulse metadata in body: { pulse_id, expires_at, trust_level }
```

19. Full REST API

```
POST /sparxstar/v1/context       → initialize or restore context
POST /sparxstar/v1/device        → register or update device record
GET  /sparxstar/v1/identity      → resolve current identity
GET  /sparxstar/v1/session       → session status
POST /sparxstar/v1/pulse         → request a fresh Context Pulse for Helios
POST /sparxstar/v1/client-report → telemetry and error reporting
```

20. UEC Compatibility and Migration

The original sparxstar-user-environment-check plugin is in production. Sirus replaces UEC entirely with a namespace shim that preserves the full public contract. The following signatures are frozen and must never change:

```
namespace Starisian\SparxstarUEC;

final class StarUserEnv
{
    public static function get_browser_name(): ?string
    { return ContextEngine::current()?->environment->browser_name; }

    public static function get_os(): ?string
    { return ContextEngine::current()?->environment->os; }

    public static function get_device_type(): ?string
    { return ContextEngine::current()?->environment->device_type; }

    public static function get_network_effective_type(): ?string
    { return ContextEngine::current()?->environment->network_effective_type; }

    public static function get_ip_address(): ?string
    { return ContextEngine::current()?->environment->ip_address; }

    public static function get_location(): ?array
    { return ContextEngine::current()?->environment->location; }
}
```

20.1 Migration Strategy — Transparent Proxy

UEC is active in production. Migration follows the Transparent Proxy pattern per Platform Integrity Map Decision 5.6:

1. Sirus boots as mu-plugin and populates its internal environment record on every request.
2. The UEC shim proxies all existing method calls to Sirus internals. No calling code changes.
3. For the first 30 days post-deployment, fail-closed governance rules apply only to new /sparxstar/ and /aiwa/ endpoints. Existing UEC-consuming code paths continue to function.
4. After the 30-day stabilisation window, fail-closed rules extend to all governed paths. The UEC shim remains permanently as the frozen public interface.

21. Subsystem Structure

```
Sirus
├── Context Engine         ← core runtime, produces context on every request
├── Context Cache          ← prevents redundant resolution within a request
├── Identity Resolver      ← five-tier identity resolution
├── Device Continuity      ← server-issued device_id, fingerprint, session recovery
├── Environment Resolver   ← browser, OS, network, location via Matomo
DeviceDetector
```

```
├── Trust Engine           ← trust state, trust score, anomaly detection
├── Consent Manager        ← technical consent, purpose consent, consent history
├── Authority Resolver     ← governance scope, multi-authority aggregation
├── Pulse Generator        ← produces signed Context Pulse for Helios
├── Pulse Verifier         ← six-check canonical verification contract
├── Device Matcher         ← fingerprint scoring thresholds for session recovery
├── Capability Engine      ← named capability issuance from context
├── Network Awareness      ← multisite, alias domains, cross-site identity
├── Network Context Broker ← portable signed context token, cross-domain handoff
├── REST API Layer         ← six endpoints including /sparxstar/v1/pulse
├── Client Telemetry       ← error reporting, aggregation, pruning
└── UEC Compatibility      ← frozen public interface, Sirus-powered internals
```

22. Open Items

The following items remain open. All other previously open items are closed per the Platform Integrity Map v1.0.

- [OPEN] Pulse TTL exact values: specific durations per connectivity tier require real-world testing in African low-connectivity environments. Working defaults are defined in Section 11.
- [OPEN] Learned trust patterns: whether and how Sirus learns normal behaviour for a device/identity pair and flags deviations. Architecturally desirable, not yet specified.
- [OPEN] Multi-Briefcase federation: how Sirus context travels between distinct Briefcase deployments in direct federation.

23. Absolute Prohibitions

- Sirus must never authenticate users — that belongs to Helios
- Sirus must never call wp_set_auth_cookie() or issue JWTs
- Sirus must never expose Helios REST endpoints to client JavaScript
- Sirus must never query Dheghom, Starmus, or any external plugin directly
- Context tokens must never contain identity claims — environment context only
- device_id must always be server-issued — never derived from JS fingerprint alone
- IP addresses must never be stored with the full last octet
- Location must never be stored at exact coordinate precision without explicit grant
- Client telemetry must never be stored in WordPress post meta
- The JS client must never grow beyond lightweight initialization and signal capture
- The UEC compatibility shim signatures must never change
- PHPStan Level 5 must pass before any merge to main
- ContextEngine::current() consumers must always use the null-safe operator (?->)
- The ContextPulse must never be used as a source of identity_id — use the full SirusContext or Helios-verified request context

24. Code Generation Order

5. EnvironmentRecord — DTO, privacy rules enforced at construction
6. DeviceRecord — DTO, activity window logic
7. SirusContext — core context DTO, multi-authority, multi-sig support
8. ContextPulse — signed pulse DTO for Helios consumption
9. VerificationResult — enum: VALID, EXPIRED, TAMPERED, MALFORMED
10. MatchResult — enum: STRONG_MATCH, WEAK_MATCH_REQUIRE_STEP_UP, NO_MATCH
11. ContextCache — static request-scoped context store
12. ContextEngine — context creation and current() accessor, CLI System Context
13. TrustEngine — trust state, trust score computation per Decision 5.1
14. DeviceContinuity — server-issued device_id, fingerprint, session recovery
15. DeviceMatcher — fingerprint scoring thresholds (Section 14.3)
16. EnvironmentResolver — Matomo DeviceDetector integration
17. PulseGenerator — produces signed ContextPulse, manages TTL strategy
18. PulseVerifier — six-check canonical verification contract (Section 13)
19. StepUpPolicy — step-up logic for Level 2 (trust < 0.7) and Level 3 (always)
20. ConsentManager — technical consent, purpose consent, consent history
21. IdentityResolver — five-tier identity resolution
22. AuthorityResolver — governance scope resolution, multi-authority aggregation
23. CapabilityEngine — named capability issuance from context
24. NetworkAwareness — multisite and alias domain handling
25. NetworkContextBroker — signed portable token, cross-domain handoff, localStorage re-hydration
26. HeliosIntegration — trust cache + internal trust resolution client
27. REST API — all six endpoints including /sparxstar/v1/pulse
28. ClientTelemetry — report table, aggregation table, pruning job, fingerprinting
29. UEC Compatibility Shim — StarUserEnv class and filters
30. JS Client — IIFE, device_id management, context init, error capture

---

Sirus is the platform kernel. Every other component depends on the context it establishes. It runs first. It cannot be deactivated. Everything downstream trusts what it produces.